



DulceByte

Proyecto final MOD 3.3, 3.4, 3.5 - Fase 2

Documentación de la API REST

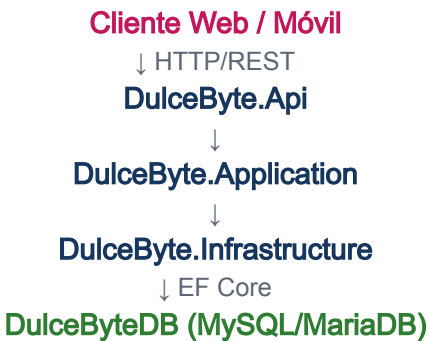
Estudiante	Melanie Hernández
Docente	Daniel Sosa
Emprendimiento	DulceByte - Cafetería y Repostería
Tecnologías	ASP.NET Core 8, Entity Framework Core, MySQL/MariaDB, Swagger/OpenAPI
Fecha	1 de septiembre de 2026

1. Descripción general

La Fase 2 implementa una API REST para DulceByte utilizando ASP.NET Core. La API funciona como capa intermedia entre la base de datos relacional y las aplicaciones web y móvil de las fases posteriores. Permite consultar y modificar información del negocio mediante endpoints HTTP documentados con Swagger/OpenAPI.

2. Arquitectura de la solución

- DulceByte.Api: controladores REST, configuración HTTP, Swagger y middleware global de errores.
- DulceByte.Application: servicios, DTOs e interfaces utilizadas por los controladores.
- DulceByte.Domain: entidades del negocio, interfaces de repositorio y excepciones de dominio.
- DulceByte.Infrastructure: DbContext de Entity Framework Core y repositorios de acceso a datos.



3. Configuración y tecnologías

Componente	Implementación
Framework	ASP.NET Core 8 / .NET 8
Persistencia	Entity Framework Core + proveedor MySQL
Base de datos	MySQL/MariaDB - esquema dulcebytedb
Documentación	Swagger / OpenAPI 3.0
Acceso HTTP	Controllers REST + CORS habilitado
Errores	ExceptionHandlerMiddleware + excepciones de dominio

4. Catálogo de endpoints

Los siguientes endpoints fueron identificados en Swagger y corresponden a las necesidades reales del emprendimiento.

4.1 Categorías

Método	Endpoint	Parámetro / Body	Uso	HTTP esperado
GET	/api/categorias	-	Listar categorías	200
POST	/api/categorias	JSON categoría	Registrar categoría	201/200
GET	/api/categorias/{id}	id	Consultar categoría	200 / 404
PUT	/api/categorias/{id}	id + JSON	Modificar categoría	200 / 404
DELETE	/api/categorias/{id}	id	Eliminar categoría	204 / 404

4.2 Clientes

Método	Endpoint	Parámetro / Body	Uso	HTTP esperado
GET	/api/clientes	-	Listar clientes	200
POST	/api/clientes	JSON cliente	Registrar cliente	201/200
GET	/api/clientes/{id}	id	Consultar cliente	200 / 404
PUT	/api/clientes/{id}	id + JSON	Modificar cliente	200 / 404

4.3 Productos

Método	Endpoint	Parámetro / Body	Uso	HTTP esperado
GET	/api/productos	idCategoria, disponible (opcionales)	Listar / filtrar productos	200
POST	/api/productos	JSON producto	Registrar producto	201
GET	/api/productos/{id}	id	Consultar producto	200 / 404
PUT	/api/productos/{id}	id + JSON	Modificar producto	200 / 404
DELETE	/api/productos/{id}	id	Eliminar producto	204 / 404

4.4 Estados de pedido

Método	Endpoint	Parámetro / Body	Uso	HTTP esperado
GET	/api/estados-pedido	-	Listar estados disponibles	200

4.5 Pedidos

Método	Endpoint	Parámetro / Body	Uso	HTTP esperado
GET	/api/pedidos	-	Listar pedidos con detalle	200
POST	/api/pedidos	{ idCliente }	Crear pedido	201
GET	/api/pedidos/cliente/{idCliente}	idCliente	Listar pedidos del cliente	200
GET	/api/pedidos/{id}	id	Consultar pedido	200 / 404
POST	/api/pedidos/{id}/detalle	id + { idProducto, cantidad }	Agregar producto al pedido	200
PUT	/api/pedidos/{id}/estado	id + { idEstado }	Actualizar estado	200

5. Validaciones y manejo de errores

- Las entidades y DTOs son procesados mediante servicios y repositorios antes de persistirse.
- La API utiliza un middleware global de excepciones (ExceptionHandlerMiddleware).
- El dominio incluye excepciones específicas como NotFoundException y BusinessException.
- Se utilizan códigos HTTP adecuados: 200 para consultas/modificaciones exitosas, 201 para creación y 204 para eliminación sin contenido.
- La base de datos mantiene restricciones e integridad referencial, complementando las validaciones de la API.

6. Swagger / OpenAPI

Swagger se habilita desde Program.cs mediante AddSwaggerGen(), UseSwagger() y UseSwaggerUI(). Durante las pruebas la interfaz estuvo disponible en <http://localhost:5080/swagger> y permitió ejecutar los endpoints de Categorías, Clientes, EstadosPedido, Pedidos y Productos.

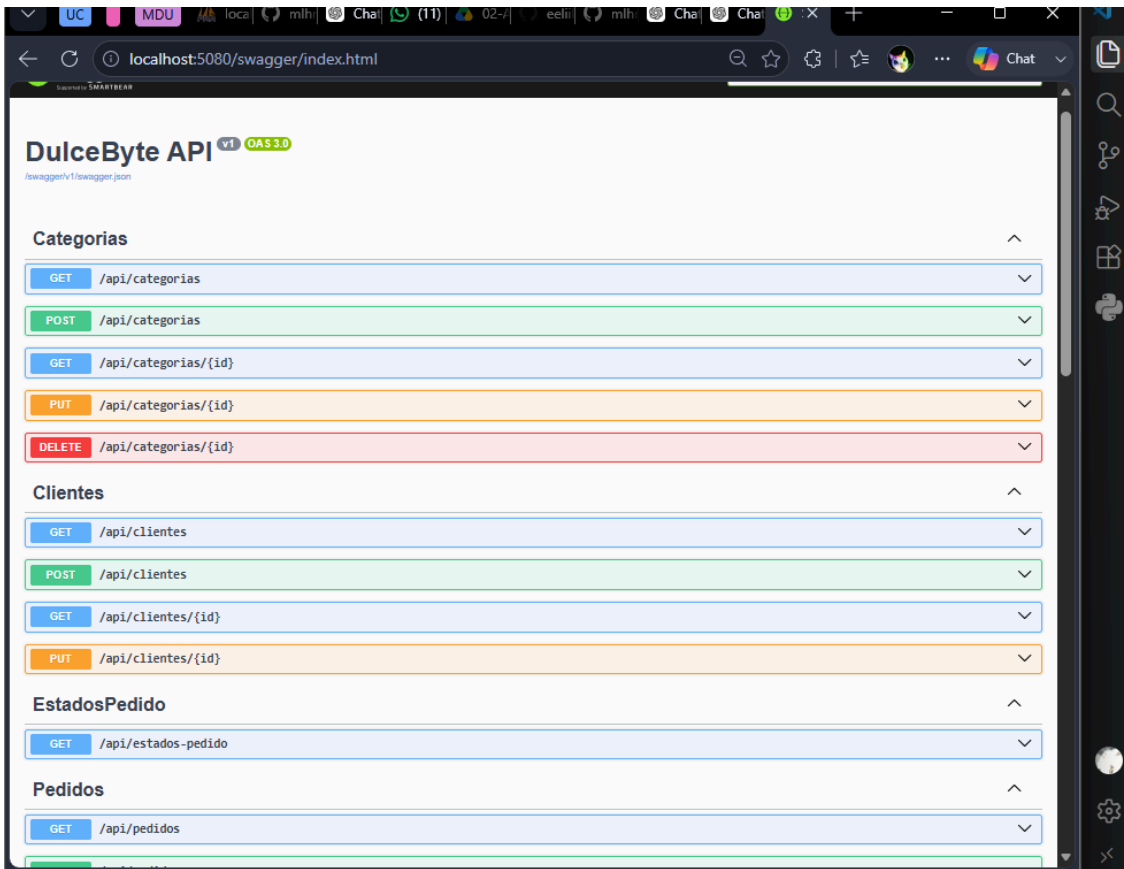


Figura 1. Interfaz Swagger de DulceByte API v1.

7. Resultado de verificación técnica

Requisito	Estado
Arquitectura por capas	Cumple
Endpoints REST	Cumple
CRUD	Cumple
Conexión con BD	Cumple
Manejo de errores	Cumple
Validación de datos	Implementada mediante capas de aplicación/dominio y BD
Respuestas HTTP	Cumple
Swagger/OpenAPI	Cumple

8. Conclusión

La API REST de DulceByte se encuentra operativa y conectada a la base de datos del proyecto. La estructura por capas permite separar responsabilidades y facilita que las aplicaciones Spring y móvil consuman los mismos servicios REST en las fases siguientes.